

An *object* is a representation of a piece of data, such as a number, some text, or a more complicated data structure, such as a list or dictionary. All objects can be stored in variables, passed as arguments to function calls, and returned from function calls.

All objects have a value, identity, and data type. The *value* is the data the object represents, such as the integer 42 or the string 'hello'. Although somewhat confusing, some programmers use the term value as a synonym for *object*, especially for simple data types like integers or strings. For example, a variable that contains 42 is a variable that contains an integer value, but we can also say it's a variable that contains an integer object with a value of 42.

An object is created with an *identity* that is a unique integer you can view by calling the `id()` function. For example, enter the following code into the interactive shell:

---

```
>>> spam = ['cat', 'dog', 'moose']
>>> id(spam)
33805656
```

---

The variable `spam` stores an object of the list data type. Its value is `['cat', 'dog', 'moose']`. Its identity is 33805656, although the integer ID varies each time a program runs so you'll likely get a different ID on your computer. Once created, an object's identity won't change for as long as the program runs. Although the data type and the object's identity will never change, an object's value can change, as we'll see in this example:

---

```
>>> spam.append('snake')
>>> spam
['cat', 'dog', 'moose', 'snake']
>>> id(spam)
33805656
```

---

Now the list also contains 'snake'. But as you can see from the `id(spam)` call, its identity hasn't changed and it's still the same list. But let's see what happens when you enter this code:

---

```
>>> spam = [1, 2, 3]
>>> id(spam)
```

---